Html5 apis and forms

1B. HTML5 APIs & Forms

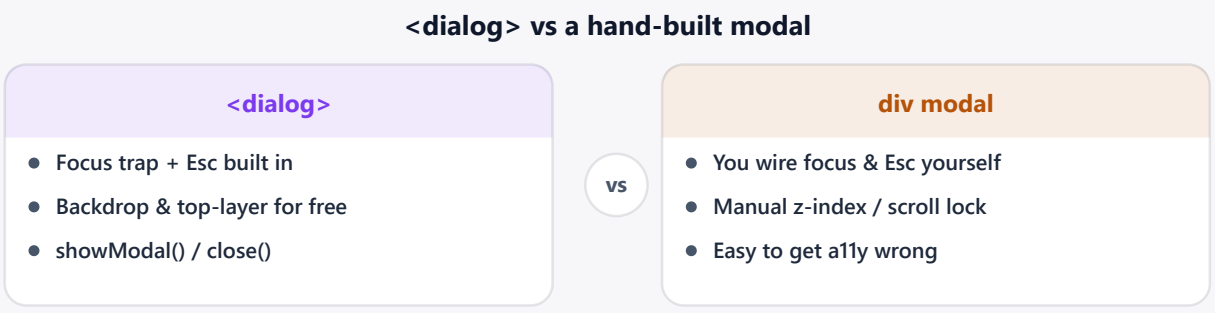
Questions: 4

Difficulty: Mid

Companies: Google, Microsoft, Flipkart, Razorpay, Cred, PhonePe, Amazon

These are the questions that separate candidates who "know HTML" from candidates who actually build with the platform. Using `<dialog>` in a machine coding round, reaching for `FormData` instead of manually reading inputs, understanding native validation before adding a library — these signal experience.

Q 1.5 Explain the HTML5 `<dialog>` element. Why should we prefer it over custom modals?



The native <dialog> hands you accessibility that a div modal makes you reinvent.

Difficulty: Mid

Asked at: Google, Microsoft, Cred, Flipkart

bunch of JavaScript to handle keyboard events. Most implementations got at least one thing wrong. The backdrop didn't cover scroll. Focus escaped to the page behind. Escape key didn't work. Screen readers couldn't tell it was a modal.

The `<dialog>` element gives you all of that for free:

HTML Copy

```
<dialog id="confirm-dialog">
  <h2>Delete this item?</h2>
  <p>This action cannot be undone.</p>
  <form method="dialog">
    <button value="cancel">Cancel</button>
    <button value="confirm">Confirm</button>
  </form>
</dialog>

<button onclick="document.getElementById('confirm-dialog').showModal()">
  Delete
</button>
```

When you call `.showModal()`, the browser automatically handles:

Backdrop. A `::backdrop` pseudo-element appears behind the dialog, covering the entire page. You style it with CSS:

CSS Copy

```
dialog::backdrop {
  background: rgba(0, 0, 0, 0.5);
  backdrop-filter: blur(4px);
}
```

Focus trapping. Tab and Shift+Tab cycle through focusable elements inside the dialog only. Focus doesn't escape to the page behind. You don't need to write a single line of focus management code.

Accessibility. The dialog implicitly gets `role="dialog"` and `aria-modal="true"`. Screen readers announce it as a dialog and know the rest of the page is inert.

Top layer. The dialog is placed in the browser's top layer — above everything else on the page. No `z-index` wars. No worrying about stacking contexts.

The `method="dialog"` trick is the elegant part. When a button inside a form with `method="dialog"` is clicked, the dialog closes automatically and the button's `value` is available through `dialog.returnValue`:

JAVASCRIPT Copy

```
const dialog = document.getElementById('confirm-dialog');

dialog.addEventListener('close', () => {
  if (dialog.returnValue === 'confirm') {
    deleteItem();
  }
  // dialog.returnValue is 'cancel' if Cancel was clicked
});
```

No need for separate click handlers on each button. No need to track which button was clicked. The form submission closes the dialog and passes the value.

Opening modally vs non-modally:

JAVASCRIPT Copy

```
// Modal: backdrop, focus trap, escape-to-close
dialog.showModal();

// Non-modal: no backdrop, no focus trap, no escape handling
dialog.show();
```

`.show()` opens the dialog as a non-modal — like a popover or an info panel. The user can still interact with the rest of the page. `.showModal()` is the full modal experience.

Animating the dialog:

CSS

Copy

```css
dialog {
  opacity: 0;
  transform: translateY(-20px);
  transition: opacity 0.2s ease, transform 0.2s ease,
              display 0.2s ease allow-discrete;
}

dialog[open] {
  opacity: 1;
  transform: translateY(0);
}

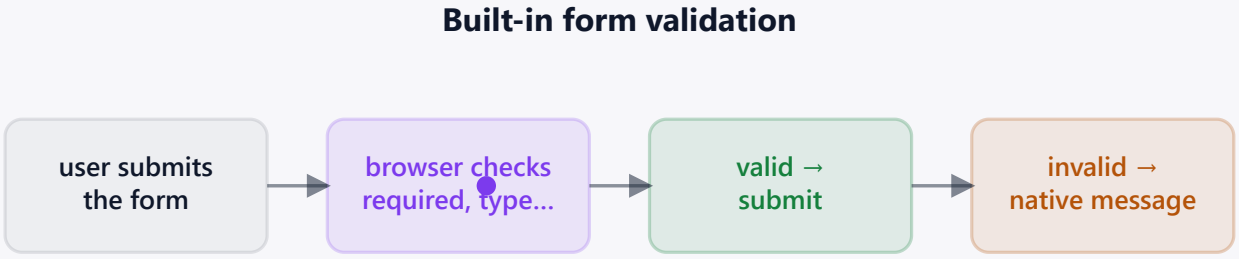
/* Starting style for the animation when opening */
@starting-style {
  dialog[open] {
    opacity: 0;
    transform: translateY(-20px);
  }
}
```

Why this matters in interviews: Machine coding rounds often ask you to build a modal, confirmation dialog, or alert. Using `<dialog>` shows you know the platform. Building a custom modal from scratch shows you can code — but not using `<dialog>` when asked "how would you improve this?" shows you don't keep up with browser capabilities. The interviewer wants to see both skills.

Follow-up: "Are there any limitations of `<dialog>` that would make you go custom?"

A few edge cases. Animation on close is tricky (the dialog is removed from the top layer immediately). Nested modals (a dialog opening another dialog) work but need careful management. And if you need a non-rectangular modal or a modal that appears from a specific position (like a slide-in panel), you might need custom positioning logic on top of `<dialog>`.

validation?



Constraint attributes (required, type, pattern, min) validate for free before any JS runs — enhance with JS, do not replace it.

Difficulty: Mid

Asked at: PhonePe, Flipkart, Amazon, Razorpay

HTML has a full constraint validation API built into form elements that most developers never explore because they jump straight to Formik or React Hook Form. Understanding what the browser provides natively is the baseline interviewers expect.

Built-in validation attributes:

HTML Copy

```
<form id="signup-form">
  <div>
    <label for="username">Username</label>
    <input
      type="text"
      id="username"
      name="username"
      required
      minlength="3"
      maxlength="20"
      pattern="^[a-zA-Z0-9_]+$"
      title="Only letters, numbers, and underscores"
    />
  </div>

  <div>
```

```
      id="email"
      name="email"
      required
      placeholder="you@company.com"
    />
  </div>

  <div>
    <label for="password">Password</label>
    <input
      type="password"
      id="password"
      name="password"
      required
      minlength="8"
      pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).+"
      title="Must contain at least one uppercase, one lowercase, and one n
    />
  </div>

  <div>
    <label for="age">Age</label>
    <input
      type="number"
      id="age"
      name="age"
      min="18"
      max="120"
      step="1"
    />
  </div>

  <button type="submit">Sign Up</button>
</form>
```

The browser checks `required`, `minlength`, `maxlength`, `pattern`, `type` (email validates email format, url validates URLs), `min`, `max`, `step` — and prevents form submission if any check fails. It renders native error tooltips, highlights invalid fields, and announces errors to screen readers. Zero JavaScript.

The Constraint Validation API — programmatic control:

When you need custom error messages or want to validate beyond what attributes offer, use the JavaScript API:

JAVASCRIPT Copy

```
const form = document.getElementById('signup-form');
const email = form.querySelector('#email');
const password = form.querySelector('#password');

email.addEventListener('input', () => {
  if (email.validity.typeMismatch) {
    email.setCustomValidity('Please enter a valid email address');
  } else if (email.validity.valueMissing) {
    email.setCustomValidity('Email is required to create your account');
  } else {
    email.setCustomValidity('');  // Clear the custom error
  }
});

form.addEventListener('submit', (e) => {
  if (!form.checkValidity()) {
    e.preventDefault();
    form.reportValidity();  // Show the native validation UI
    return;
  }

  // All valid — submit the form
  submitForm(new FormData(form));
});
```

The `validity` object on every input has boolean properties that tell you exactly what's wrong:

JAVASCRIPT Copy

```
input.validity.valueMissing     // required field is empty
input.validity.typeMismatch     // type="email" but value isn't an email
input.validity.patternMismatch  // doesn't match the pattern attribute
input.validity.tooShort         // shorter than minlength
input.validity.tooLong          // longer than maxlength
```

```
input.validity.customError      // setCustomValidity() was called
input.validity.valid            // all checks pass
```

CSS pseudo-classes for styling invalid fields:

css Copy

```css
/* Style invalid fields */
input:invalid {
  border-color: #e53e3e;
}

/* Only show invalid style after user has interacted */
input:invalid:not(:placeholder-shown) {
  border-color: #e53e3e;
  background: #fff5f5;
}

/* Valid field */
input:valid {
  border-color: #38a169;
}

/* The :user-invalid pseudo-class (newer) — only shows invalid after user
input:user-invalid {
  border-color: #e53e3e;
}
```

When to use native validation:

Simple forms with straightforward rules — login forms, search bars, contact forms, newsletter signups. Adding a library for a 3-field form is engineering overhead that isn't justified.

When to use JavaScript libraries (React Hook Form, Formik + Yup/Zod):

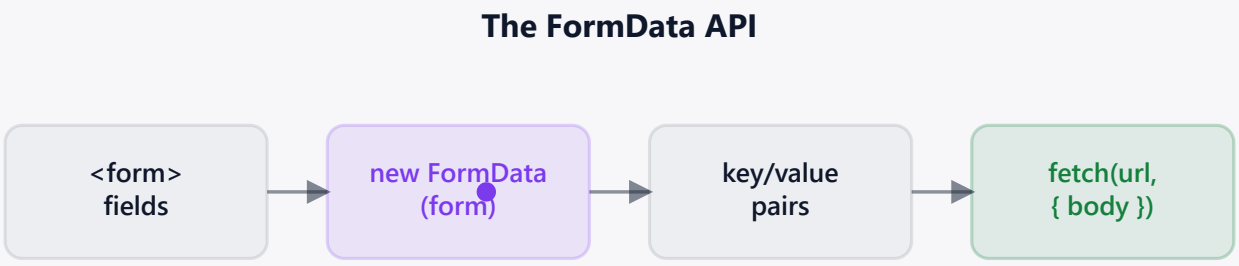
- Multi-step forms where validation spans across steps

- Async validation (check if username is already taken via API)
- Complex conditional logic (field X is required only if field Y has value Z)
- When you need consistent styling across browsers (native validation UI differs between Chrome, Firefox, Safari)

Follow-up: "What does `novalidate` on the form element do?"

`novalidate` disables the browser's built-in validation UI — the tooltips and prevention of form submission. But it doesn't disable the validation API. You can still call `checkValidity()`, `reportValidity()`, and read `input.validity` in JavaScript. This gives you full control over when and how validation errors are displayed, which is useful when you want custom-styled error messages instead of the browser's default tooltips.

Q 1.7 What's the `FormData` API and why is it useful?



FormData scoops every named field (files included) into one object you can send straight to fetch — no manual serialization.

Difficulty: Mid

Asked at: Razorpay, Swiggy, Uber, Amazon

`FormData` is a built-in browser API that automatically captures all form field values into a key-value structure. You don't need to manually query each input, read its value, and assemble an object. The form already knows what its fields are.

Basic usage:

```javascript
const form = document.querySelector('#checkout-form');

form.addEventListener('submit', async (e) => {
  e.preventDefault();

  const formData = new FormData(form);

  // Get individual values
  const email = formData.get('email');         // string or null
  const plan = formData.get('plan');           // string or null

  // Get all values for multi-select or checkbox groups
  const interests = formData.getAll('interests');  // string[]

  // Convert to a plain object (simple cases)
  const data = Object.fromEntries(formData);
  // { email: "ankit@example.com", plan: "pro" }

  // Send as JSON
  await fetch('/api/checkout', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });
});
```

Where FormData shines — file uploads:

This is the killer use case. When you pass a `FormData` object as the `fetch` body, the browser automatically sets the `Content-Type` to `multipart/form-data` with the correct boundary. You don't need to manually serialize files or set headers.

JAVASCRIPT Copy

```javascript
const formData = new FormData();
formData.append('avatar', fileInput.files[0]);
formData.append('username', 'ankit');
formData.append('bio', 'Frontend engineer');

// The browser handles Content-Type: multipart/form-data automatically
```

```
  // Do NOT set Content-Type header — the browser sets it with the correct
});
```

Common mistake: setting `Content-Type: multipart/form-data` manually. The browser needs to generate a boundary string and include it in the header. If you set the header yourself, the boundary is missing and the server can't parse the request.

Creating FormData without a form element:

JAVASCRIPT Copy

```javascript
const formData = new FormData();
formData.append('name', 'Ankit');
formData.append('email', 'ankit@example.com');

// You can also append files directly
formData.append('document', blob, 'report.pdf');
```

Iterating over FormData:

JAVASCRIPT Copy

```javascript
const formData = new FormData(form);

// Iterate all entries
for (const [key, value] of formData.entries()) {
  console.log(`${key}: ${value}`);
}

// Check if a field exists
formData.has('email');  // true or false

// Modify entries
formData.set('email', 'new@example.com');  // Replaces existing value
formData.append('tag', 'javascript');       // Adds another value (doesn't
formData.delete('temporary_field');         // Removes the entry
```

In React — using FormData with form actions:

JSX

Copy

```jsx
function ContactForm() {
  async function handleSubmit(e) {
    e.preventDefault();
    const formData = new FormData(e.currentTarget);
    const data = Object.fromEntries(formData);

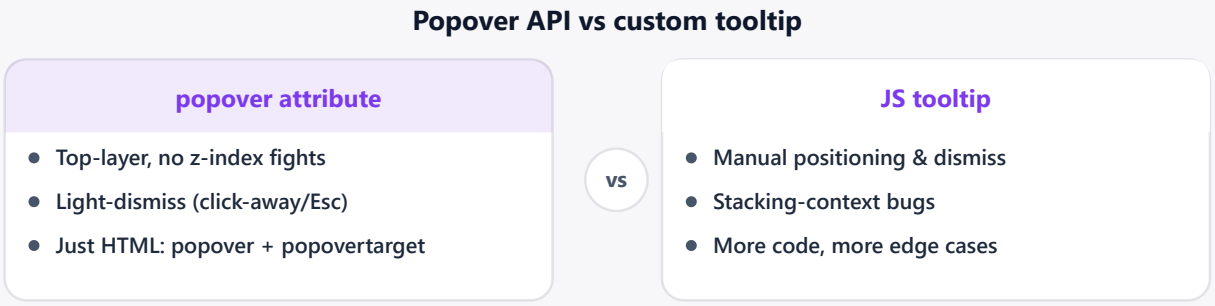
    await fetch('/api/contact', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data),
    });
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" required />
      <input name="email" type="email" required />
      <textarea name="message" required />
      <button type="submit">Send</button>
    </form>
  );
}
```

Notice there's no `useState` for each field. No `onChange` handlers tracking every keystroke. The form elements manage their own state natively (uncontrolled components), and `FormData` reads everything at submission time. For forms that don't need real-time validation or conditional logic, this is significantly simpler than the controlled component pattern.

Why interviewers ask this: In machine coding rounds, when you're building a form-heavy feature (checkout flow, multi-step wizard, settings page), using `FormData` shows you think about simplicity and reach for the right tool. Tracking every field in React state is the default approach most candidates take — but for many forms, it's unnecessary complexity.

Q 1.8 What is the Popover API and how does it compare to tooltips and modals?



Popovers are non-modal (they do not block the page); a <dialog> is for modal, must-answer interactions.

Difficulty: Mid

Asked at: Google, Microsoft, Amazon

The Popover API is a browser-native way to create popovers — elements that appear on top of other content. Dropdown menus, action menus, date picker panels, tooltip-like information boxes. Before this API, every one of these required custom JavaScript for showing/hiding, click-outside-to-close, escape-to-close, and z-index management.

Basic usage — no JavaScript required:

HTML Copy

```html
<!-- The trigger button -->
<button popovertarget="actions-menu">Actions</button>

<!-- The popover content -->
<div id="actions-menu" popover>
  <button>Edit</button>
  <button>Duplicate</button>
  <button>Archive</button>
  <button>Delete</button>
</div>
```

That's it. Click the button, the popover appears. Click it again, it hides. Click anywhere outside, it hides. Press Escape, it hides. The browser handles all of this.

What the browser provides automatically:

- Toggle visibility on button click
- Light dismiss — clicking outside closes the popover
- Escape key closes it
- Top layer placement — no z-index issues, no overflow clipping from parent containers
- Accessibility announcements
- No scroll-lock on the page (unlike modals)

The `popover` attribute has two modes:

`popover="auto"` **(default):** Light dismiss behavior. Clicking outside closes it. Only one auto popover can be open at a time — opening a second one closes the first. This is the behavior you want for dropdown menus and action menus.

`popover="manual"`**:** No light dismiss. The popover stays open until explicitly closed via JavaScript. Multiple manual popovers can coexist on the page. This is the behavior you want for toast notifications or persistent info panels.

HTML Copy

```html
<!-- Auto: one at a time, closes on click outside -->
<div id="dropdown" popover="auto"> ... </div>

<!-- Manual: stays until you close it programmatically -->
<div id="toast" popover="manual">
  File saved successfully
</div>

<script>
  const toast = document.getElementById('toast');

  // Show programmatically
  toast.showPopover();

  // Hide after 3 seconds
  setTimeout(() => toast.hidePopover(), 3000);
</script>
```

Controlling show/hide behavior from the trigger:

```
<!-- Toggle (default) - click opens, click again closes -->
<button popovertarget="menu" popovertargetaction="toggle">Menu</button>

<!-- Show only - click opens, clicking the button again does nothing -->
<button popovertarget="info" popovertargetaction="show">Show Info</button>

<!-- Hide only - useful for a close button inside the popover -->
<button popovertarget="menu" popovertargetaction="hide">Close</button>
```

Styling the popover:

CSS Copy

```
/* The popover itself */
[popover] {
  padding: 16px;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  max-width: 300px;
}

/* The backdrop (only appears for auto popovers when using ::backdrop) */
[popover]::backdrop {
  background: transparent;  /* No visible backdrop by default */
}

/* Positioning relative to the trigger using CSS Anchor Positioning */
[popover] {
  position-anchor: --trigger;
  top: anchor(bottom);
  left: anchor(left);
}
```

How Popover compares to `<dialog>` and custom tooltips:

Feature	Popover API	`<dialog>` (modal)	Custom tooltip

trapping	No	Yes	No
Light dismiss	Yes (auto mode)	No (Escape only)	Custom JS needed
Backdrop	Optional	Yes (`::backdrop`)	No
Top layer	Yes	Yes	No (z-index battles)
Escape to close	Yes	Yes	Custom JS needed
JavaScript needed	Minimal to none	`.showModal()`	Everything
Multiple open at once	Manual mode only	No (one modal at a time)	Yes
Page scroll blocked	No	No (but content behind is inert)	No

When to use which:

- **Popover API** — Dropdown menus, action menus, date picker panels, teaching/onboarding UI, lightweight info panels. Anything that should appear above content without blocking interaction with the rest of the page.

page.

- **Custom tooltip** — Only if you need precise positioning logic that CSS Anchor Positioning can't handle (rare in 2026) or need to support much older browsers.

Follow-up: "Can you nest popovers?"

Yes. An auto popover can trigger another auto popover. The browser manages a "popover stack" — clicking outside closes all of them, and pressing Escape closes the topmost one. This is useful for nested dropdown menus (e.g., a "More" menu inside an action menu).

QUICK CHECK — no marks, just for you.

1. You open a <dialog> by calling dialog.show() instead of dialog.showModal(). What behaviour should you expect?

- ○ The dialog opens non-modally: no backdrop and no focus trap, and the user can still interact with the rest of the page
- ○ The dialog opens with a backdrop and focus trap, but the Escape key no longer closes it
- ○ The dialog opens modally but is rendered in the normal stacking context, so a high z-index elsewhere can cover it
- ○ The dialog does not open at all until an open attribute is also added in HTML

Check answer

2. You are uploading a file by passing a FormData object as the body of a fetch() call, and you also set headers: { 'Content-Type': 'multipart/form-data' } yourself. Why does the server fail to parse the request?

- ○ fetch ignores any body when a Content-Type header is present, so the server receives an empty request

Setting the header manually omits the boundary string the browser would have generated, so the server cannot parse the multipart body

- The manual header forces the file to be serialized as JSON, which drops the binary content
- FormData bodies must be sent as application/x-www-form-urlencoded, so the Content-Type is simply wrong

Check answer

3. You add the novalidate attribute to a <form> because you want to render your own custom-styled error messages. What can your JavaScript still do?

- Everything in the validation API still works — you can call checkValidity() and reportValidity() and read input.validity
- Only checkValidity() works; reportValidity() and the validity object are disabled along with the native UI
- The validity object still updates, but the required, pattern and minlength attributes are no longer evaluated by the browser
- Nothing — novalidate strips the constraint attributes, so input.validity.valid is always true

Check answer

‹ Previous
Semantic structure

Next ›
Loading and resource hints

LEARN MORE

ARTICLES & DOCS

<dialog> element (MDN)
MDN

FormData API (MDN)
MDN

VIDEOS



The <dialog> element changes modals
YouTube



Exploring the HTML dialog element
YouTube



FormData API in practice
YouTube

Hand-picked references for this lesson.